COMPUTER SYSTEMS AND PLATFORM TECHNOLOGIES

IT0047

FEU ALABANG FEU DILIMAN FEU TECH

Technology Driven by Innovation

MODULE 5

8086/8088 Instruction Sets

MODULE 5.1

Data Transfer & Arithmetic Instructions

At the end of the lesson, the student should be able to :

- **Identify the different data transfer & arithmetic instructions**
- **Determine if the syntax of the given instruction is valid**
- **Derive the value of the destination operand and conditional after execution of arithmetic instructions**

Data Transfer Instructions

- Data transfer instructions facilitate the movement of data either between registers or between a register and main memory. They do not affect the flags.

MOV INSTRUCTION

Format: MOV D, S

Action: D ← [S]

Destination	Source	Example
register	register	MOV CX, BX
register	MM	MOV CX, [BP+SI]
MM	register	MOV [BX], DX
register	immediate	MOV CX, 80FEH
MM	immediate	MOV word ptr [BX], 1834H
seg reg	register	MOV DS, BX
register	seg reg	MOV AX, CS
seg reg	MM	MOV SS, [1AFFH]
MM	seg reg	MOV [BP+SI+1000H], DS

Pointers:

- The MOV instruction does not allow both source and destination operands to be memory locations (no memory to memory transfers). The instruction **MOV [BX], FILE** is invalid.
- Both source and destination operands should be of the same data size. Therefore, the instruction **MOV AH, BX** is invalid.
- Immediate data cannot be moved to a segment register. **MOV DS, 1800H** is therefore invalid. Instead, use:

```
MOV AX, 1800H
MOV DS, AX
```

Continuation

- The destination operand cannot be immediate data. **MOV 1800H, AX** is therefore invalid.
- More often than not, register CS cannot be the destination operand.
- If the destination operand is a memory location and the source operand is immediate data, the prefix ***byte ptr*** or ***word ptr*** should appear after the MOV instruction to denote the data size of the immediate operand.

XCHG INSTRUCTION

The XCHG (***Exchange***) instruction swaps the contents of the source and destination operands.

Format: XCHG D, S

Action: [D] ← [S]

Destination	Source	Example
MM	register	XCHG [BX], CX
register	register	XCHG AH, BL

Pointers:

- The XCHG instruction does not allow both source and destination operands to be memory locations (no memory to memory transfers). The instruction **XCHG [BX], FILE** is therefore invalid.
- Both source and destination operands should be of the same data size. Therefore, the instruction **XCHG AH, BX** is invalid.
- Immediate addressing cannot be used with this instruction. The instruction **XCHG AX, 1800H** is invalid.

Continuation

- A data exchange can also be done using several MOV instructions:

```
MOV  CX, AX
MOV  DX, BX
MOV  AX, DX
MOV  BX, CX
```

- However, four registers are needed in order to implement an exchange between two registers.
- Segment registers cannot be used as operands in the XCHG instruction. **XCHG ES, BX** is therefore invalid. Instead use:

```
MOV CX, ES
XCHG BX, CX
MOV ES, CX
```

PUSH INSTRUCTION

- The PUSH instruction decrements SP by 2 and then moves a word data from the source operand to the top of the stack pointed to by SP.

Format: PUSH S

Action:

$$SP \leftarrow SP - 2$$

$$[SP+1] \leftarrow [SH]$$

$$[SP] \leftarrow [SL]$$

PUSH INSTRUCTION

Source	Example
register	PUSH AX
seg reg	PUSH DS
MM	PUSH BETA

Note:

- The PUSH instruction pushes word-sized data only.

POP INSTRUCTION

- POP transfers a word data pointed to by SP to the destination operand. Afterwards, the value of SP is incremented by 2.

Format: POP D

Action:

$$DL \leftarrow [SP]$$

$$DH \leftarrow [SP + 1]$$

$$SP \leftarrow SP + 2$$

POP INSTRUCTION

Destination	Example
register	POP AX
seg reg	POP DS
MM	POP BETA

Pointers:

- The POP instruction pops word-sized data only.
- **POP CS** is not allowed. However, the instruction **PUSH CS** is valid.

LEA INSTRUCTION

The LEA (***Load Effective Address***) instruction is for transferring the effective address (not data) of the source operand to the destination operand.

Format: LEA D, S

Action: D ← EA of S

Pointers:

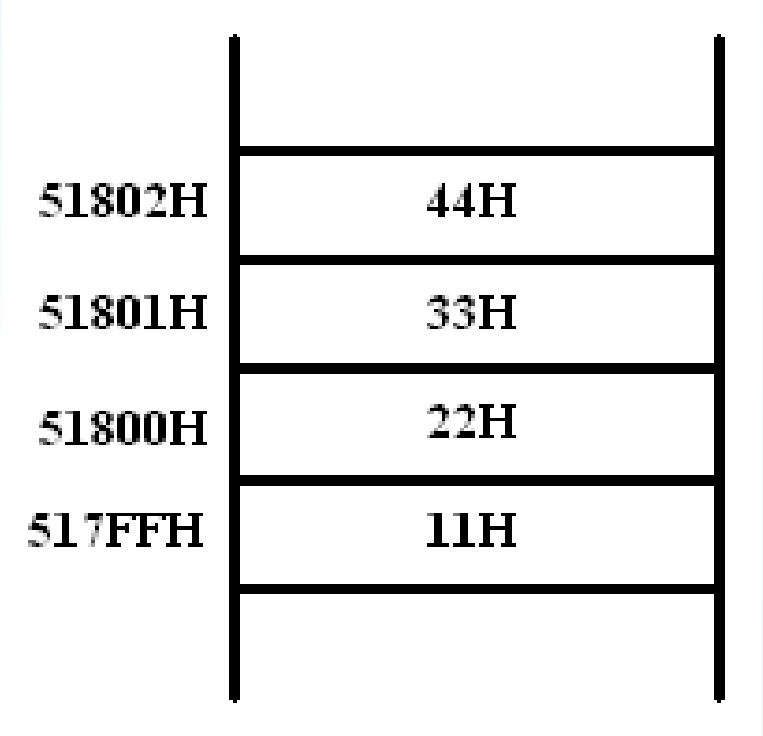
- The source operand should always be one of the memory addressing modes, while the destination operand should always be a 16-bit register.
- In the LEA instruction, the effective address is moved to the destination operand. Therefore, the destination operand is like a pointer.
- Usually, registers BX, BP, DI, or SI are used as the destination operand since these registers could be used as offset registers.

Examples:

Assume the following:

DS = 5000H

LIST = 1800H



On the Figure

- If the **MOV DI, LIST** instruction is executed, the contents of memory location 51800H and 51801H are transferred to register DI. DI will therefore contain the word 3322H.
- However, if the **LEA DI, LIST** instruction is executed, this will transfer the effective address of the source operand to register DI. Therefore, DI will now contain 1800H.

MODULE 5.1 Part 2

Basic 8086/8088 Instruction Sets: Arithmetic Instructions

At the end of the lesson, the student should be able to :

- **Identify the different 8086/8088 arithmetic instruction set**
- **Derive the value of the destination operand after execution of arithmetic instructions**
- **Derive the values of the conditional flags after execution of arithmetic instruction.**

ADD INSTRUCTION

- The ADD instruction adds the source operand to the destination operand and then places the result in the destination operand.

Format: ADD D, S

Action: D ← [D] + [S]

Destination	Source	Example
register	register	ADD BX, CX
register	MM	ADD DX, [BP + SI]
MM	register	ADD [BX + DI], CX
register	immediate	ADD BX, 0015H
MM	immediate	ADD byte ptr BETA, 12H

Pointers:

- The ADD instruction does not allow both source and destination operands to be memory locations (no memory to memory addition). Therefore, the instruction **ADD [BX], BETA** is invalid.
- Both source and destination operands should be of the same data size. The instruction **ADD AH, BX** is therefore invalid.
- The destination operand cannot be immediate data. Therefore, the instruction **ADD 1800H, CX** is invalid.

Continuation…

- All status flags are affected after execution.
- As with the MOV instruction, if the destination operand is a memory location and the source operand is immediate data, the prefix ***byte ptr*** or ***word ptr*** should appear after the ADD instruction to denote the data size of the immediate operand.

ADC INSTRUCTION

- The ADC (***Add with Carry***) instruction adds the source operand and the carry flag to the destination operand and places the result in the destination operand.

Format:	ADC D, S	
Action:	D	[D] + [S] + [CF]

- One of the uses of the ADC instruction is in the implementation of 32-bit addition (adding two 32-bit numbers).

Example:

```
      012F 8749H
+     3054 9312H
      3184 1A5BH
```

- Although there are no 32-bit addition instructions, the problem is easily solved by using the ADD and the ADC instructions.

```
MOV  AX, 012FH
MOV  BX, 8749H
MOV  CX, 3054H
MOV  DX, 9312H
ADD  BX, DX
ADC  AX, CX
```

INC INSTRUCTION

- The INC (***Increment***) instruction adds 1 to the destination operand.

Format: INC D

Action: D ← [D] + 1

Destination	Example
register	INC AX
MM	INC byte ptr [BX]

Pointers:

- The destination operand is viewed as an unsigned integer.
- The INC instruction does not affect the carry flag.
- Segment registers cannot be used as the destination operand. Therefore, the instruction **INC CS** is invalid.
- If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the INC instruction to denote the data size of the destination operand.

Binary Coded Decimal

There are only 10 combinations or values that are valid as a BCD.

VALID	INVALID
0000	101
0001	0
0010	101
0011	1
0100	110
0101	0
0110	110
0111	1
1000	1110
1001	1111

BCD Conversion:

EXAMPLE:

Each of the digit of the decimal value will be converted to BCD 4 bits value.

1. Decimal value 152
   - 000101010010
2. Decimal value 46
   - 01000110

DAA INSTRUCTION

- The DAA (***Decimal Adjust for Addition***) instruction adjusts the result of a previous addition of two valid packed decimal operands.

Format: DAA

Pointers:

- The result of the previous operation should be in register AL.
- OF is undefined after execution.
- DAA should be executed immediately after an ADD, ADC, or INC instruction.
- The instruction will adjust the result as follows: if AF = 1, or either of the nibbles is greater than 9, then 6 is added to the nibble(s) concerned.

Example:

```
MOV AL, 15H
MOV BL, 15H
ADD AL, BL
DAA
```

- The values of AL and BL are viewed as packed BCD. Since DAA is used to adjust the addition operation, the destination operand should be at AL. Prior to DAA, the value of AL is 2AH. After the DAA instruction, the value of AL is adjusted to 30H.

AAA INSTRUCTION

- The AAA (***ASCII Adjust for Addition***) instruction adjusts the result of a previous addition of two valid unpacked decimal operands.

Format: AAA

Pointers:

- The result of the previous operation should be in register AL.
- OF, PF, SF, and ZF are undefined after execution.
- AAA should be executed immediately after an ADD, ADC, or INC instruction.
- The instruction will adjust the result as follows: if the least significant nibble is greater than 9, then 6 is added to AL, and 1 is added to AH.
- Regardless of the value of the least significant nibble, the most significant nibble is always zeroed out.

Example:

MOV AL, 35H; ASCII value of 5

MOV BL, 34H; ASCII value of 4

ADD AL, BL

AAA

- AAA is used to adjust the addition operation, the destination operand should be at AL. Prior to AAA, the value of AL is 69H. After the AAA instruction, the value of AL is adjusted to 09H.
- If the least significant nibble not greater than 9, the only adjustment needed is to zero out the most significant nibble.

SUB Instruction

- The SUB instruction subtracts the source operand from the destination operand and places the result in the destination operand.

Format: SUB D, S

Action: D ← [D] - [S]

SBB Instruction

- The SBB (***Subtract with Borrow***) instruction subtracts the source operand and the carry flag from the destination operand and then places the result in the destination operand.

Format: SBB D, S

Action: D ← [D] - [S] - [CF]

- Just like in addition, subtracting a 32-bit number from another 32-bit number can be done with the combination of the SUB and the SBB instructions.

Example:

Ex. 3054 8312H
-012F 8749H
2F24 FBC9H

```
MOV   AX, 3054H
      MOV   BX, 8312H
      MOV   CX, 012FH
      MOV   DX, 8749H
      SUB   BX, DX
      SBB   AX, CX
```

DEC Instruction

- The DEC (***Decrement***) instruction subtracts 1 from the destination operand.

Format: DEC D

Action: D ⟵ [D] – 1

Destination	Example
register	DEC AX
MM	DEC word ptr [BX]

Pointers:

1. The destination operand is viewed as an unsigned integer.

2. The DEC instruction does not affect the carry flag.

3. Segment registers cannot be used as the destination operand. Therefore, the instruction **DEC CS** is invalid.

4. If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the DEC instruction to denote the data size of the destination operand.

NEG Instruction

- The NEG (***Negate***) instruction converts the specified operand to its 2's complement equivalent and the result returned to the operand location. This is, in effect, reversing the sign of an integer.

Format: NEG D

Action: D ← 0 - [D]

Destination	Example
register	NEG AX
MM	NEG byte ptr [BX]

Pointers:

1. Attempting to negate an operand having a value of zero causes no change to the operand and resets the carry flag (CF = 0).

2. Attempting to negate an operand having a value of either 80H or 8000H causes no change to the operand and sets the overflow flag (OF = 1).

3. Segment registers cannot be used as the destination operand. Therefore, the instruction **NEG CS** is invalid.

4. If the destination operand is a memory location, the prefix ***byte ptr*** or ***word ptr*** should appear after the NEG instruction to denote the data size of the destination operand.

Example:

- Determine the value of AL and the value of the flags following the instruction sequence:

AL contains:

```
      0000 0000
-     0000 0101
  ---------------
      1111 1011 = FBH = - 5
```

The flags are affected as follows:

CF = AF = SF =1, PF = ZF = OF = 0

DAS Instruction

- The DAS (***Decimal Adjust for Subtraction***) instruction adjusts the result of a previous subtraction of two valid packed decimal operands.

Format: DAS

Pointers:

1. The result of the previous operation should be in register AL.
2. OF is undefined after execution.
3. DAS should be executed immediately after a SUB, SBB, or DEC instruction.
4. The instruction will adjust the result as follows: if AF = 1, or either of the nibbles is greater than 9, then 6 is subtracted from the nibble(s) concerned.

Example:

MOV AL, 34H
MOV BL, 19H
SUB AL, BL
DAS

34H = 0011 0100
19H = 0001 1001
0001 1011 = 1BH
- 0110
0001 0101 = 15H

AAS Instruction

- The AAS (***ASCII Adjust for Subtraction***) instruction adjusts the result of a previous subtraction of two valid unpacked decimal operands.

Format: AAS

Pointers:

1. The result of the previous operation should be in register AL.
2. OF, PF, SF, and ZF are undefined after execution.
3. AAS should be executed immediately after a SUB, SBB, or DEC instruction.
4. The instruction will adjust the result as follows: if the least significant nibble is greater than 9, then 6 is subtracted from AL, and 1 is subtracted from AH.
5. Regardless of the value of the least significant nibble, the most significant nibble is always zeroed out.

Example:

MOV AL, 39H; ASCII value of 9

MOV BL, 34H; ASCII value of 4

SUB AL, BL

AAS

39H = 0011 1001

34H = 0011 0100

0000 0101 = 05H

CMP Instruction

- The CMP (***Compare***) instruction subtracts the source operand from the destination operand. It then discards the result but it updates the values of all the status flags.

Format:	CMP D, S
Action:	[D] - [S]

Valid Format

Destination	Source	Example
register	register	CMP BX, CX
register	MM	CMP CX, BETA
MM	register	CMP BETA, DX
register	immediate	CMP SI, ABCDH
MM	immediate	CMP byte ptr [BX], 34H

Pointers:

1. The CMP instruction does not allow both source and destination operands to be memory locations (no memory to memory comparison). **CMP [BX], BETA** is therefore invalid.

2. Both source and destination operands should be of the same data size. Therefore, the instruction **CMP AX, CL** is invalid.

3. As with the MOV instruction, if the destination operand is a memory location and the source operand is immediate data, the prefix ***byte ptr*** or ***word ptr*** should appear after the CMP instruction to denote the data size of the immediate operand.

Program Tracing Example:

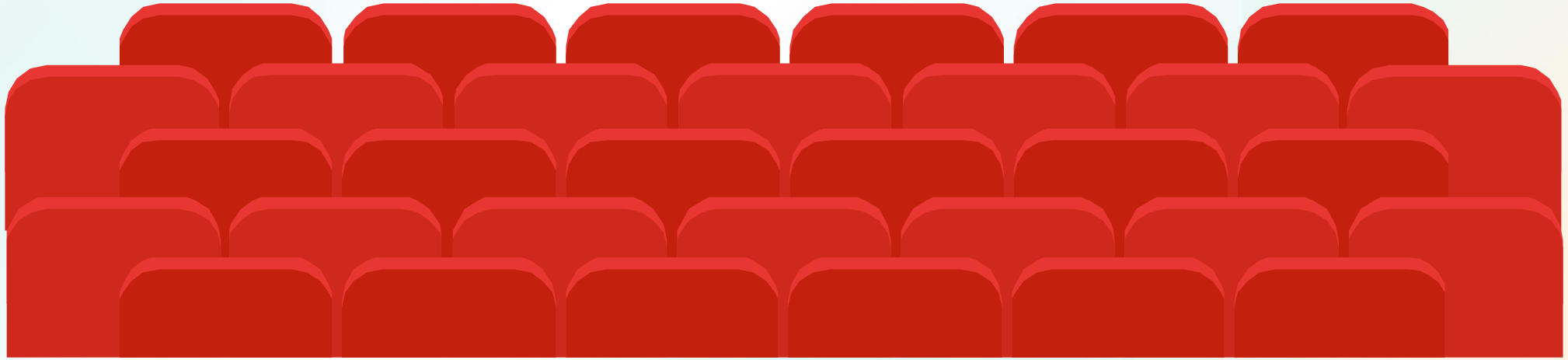
- Trace the following program. Assume the following: all flags and registers (unless specified otherwise) are initially zero, DS = 8000H, SS = 8001H, the label ALPHA = 0020H.

```
MOV AX, ALPHA
MOV SP, AX
POP BX
ADD BX, AX
SUB BX, 0020H
XCHG BX, BP
MOV CX, [BP]
SBB AX, CX
ADC AX, 0029H
ADD AL, CL
DAA
NEG AX
LEA SI, ALPHA
MOV DX, [SI]
```

Memory Map

80000H	11H	80020H	50H	80040H	85H	80060H	10H
80001H	2BH	80021H	00H	80041H	D0H	80061H	00H
80002H	3EH	80022H	0AH	80042H	7AH	80062H	59H
80003H	22H	80023H	6FH	80043H	EDH	80063H	42H
80004H	80H	80024H	A5H	80044H	47H	80064H	87H
80005H	67H	80025H	2FH	80045H	46H	80065H	50H
80006H	54H	80026H	17H	80046H	A7H	80066H	9FH
80007H	5FH	80027H	DAH	80047H	7EH	80067H	5AH
80008H	0EH	80028H	08H	80048H	96H	80068H	C0H
80009H	0FH	80029H	0BH	80049H	F2H	80069H	97H
8000AH	F4H	8002AH	2EH	8004AH	BCH	8006AH	AEH
8000BH	6BH	8002BH	CFH	8004BH	52H	8006BH	7CH
8000CH	4CH	8002CH	25H	8004CH	8BH	8006CH	1AH
8000DH	75H	8002DH	D7H	8004DH	F9H	8006DH	84H
8000EH	AAH	8002EH	39H	8004EH	53H	8006EH	2EF
8000FH	EFH	8002FH	56H	8004FH	BBH	8006FH	9EH
80010H	FDH	80030H	20H	80050H	31H	80070H	B9H
80011H	3DH	80031H	DFH	80051H	1FH	80071H	C2H
80012H	4FH	80032H	8FH	80052H	F7H	80072H	2EH
80013H	08H	80033H	44H	80053H	FFH	80073H	2DH
80014H	9AH	80034H	69H	80054H	13H	80074H	51H
80015H	49H	80035H	F6H	80055H	B1H	80075H	ADH
80016H	5DH	80036H	29H	80056H	CCH	80076H	36H
80017H	33H	80037H	5BH	80057H	16H	80077H	69H
80018H	CDH	80038H	DDH	80058H	26H	80078H	00H
80019H	F1H	80039H	ACH	80059H	92H	80079H	F9H
8001AH	0AH	8003AH	72H	8005AH	7BH	8007AH	1DH
8001BH	ABH	8003BH	F6H	8005BH	BFH	8007BH	FEH
8001CH	07H	8003CH	91H	8005CH	C7H	8007CH	61H
8001DH	62H	8003DH	20H	8005DH	B2H	8007DH	A9H
8001EH	19H	8003EH	82H	8005EH	39H	8007EH	DEH
8001FH	9EH	8003FH	DCH	8005FH	BDH	8007FH	4DH

Q&A SESSION

ASK ANY QUESTION RELATED TO OUR TOPIC FOR TODAY.

THANK YOU